

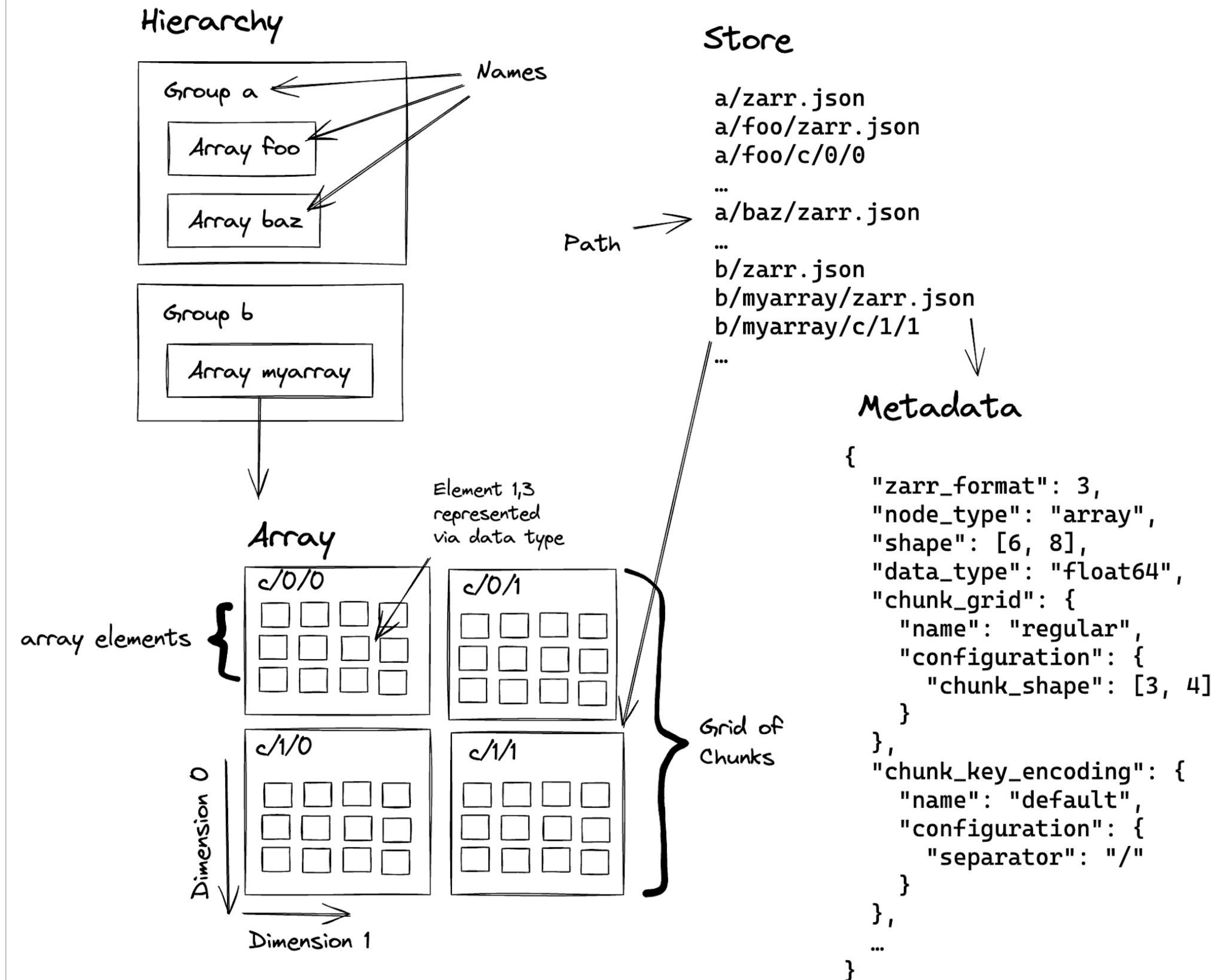


GPU Accelerated Zarr

Tom Augspurger | PyData Global 2025

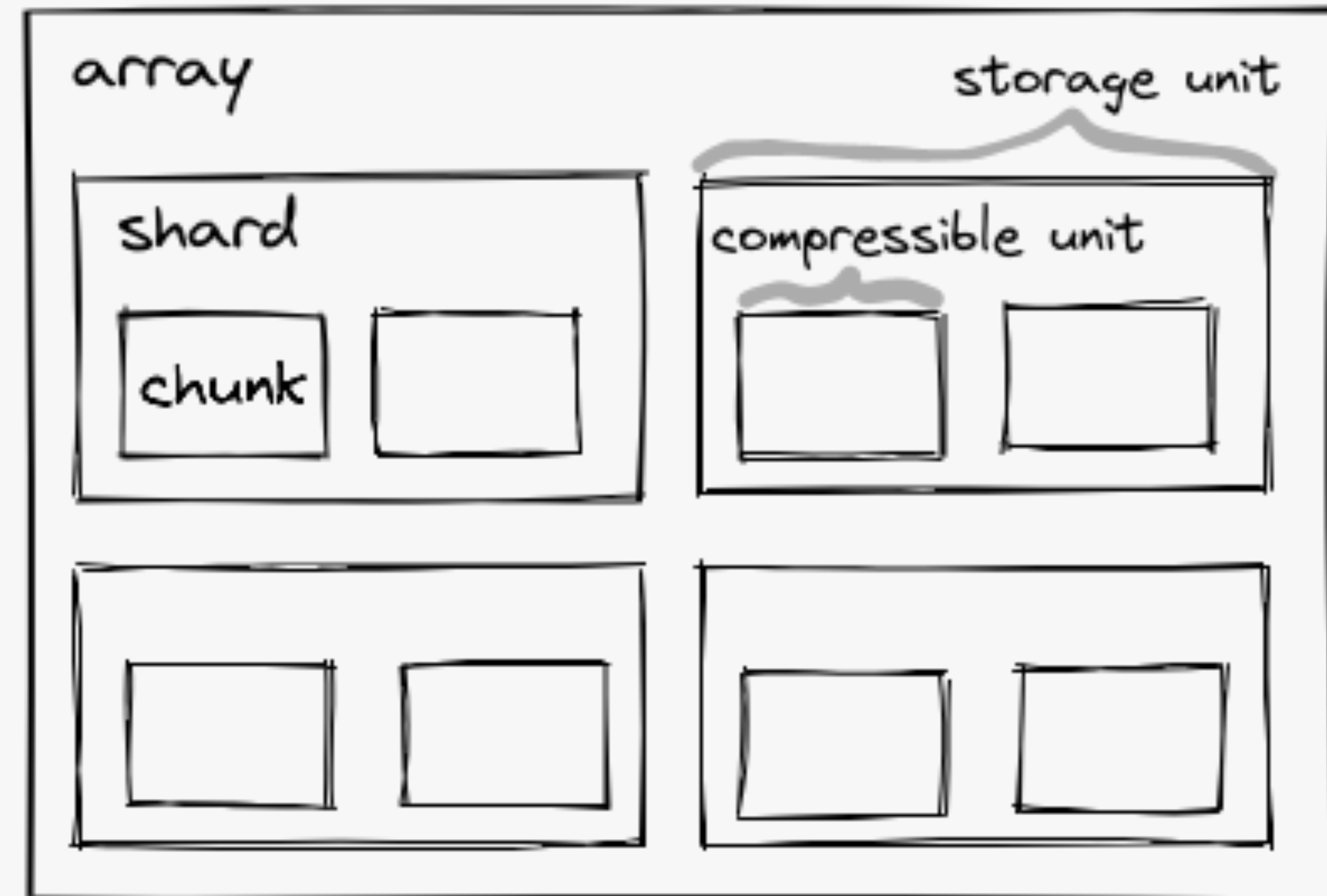
Zarr Background

- Storage format for chunked, n-dimensional arrays + metadata
- Chunks are independently compressed / encoded
- Well-suited to the cloud and HPC
- Used in many domains



Zarr Shards

- Independently compress many chunks, concatenate into a **shard**
- Great for cloud Object Storage (fewer API calls) and HPC filesystems (fewer inodes)
- Great for GPUs, since we can parallelize decoding

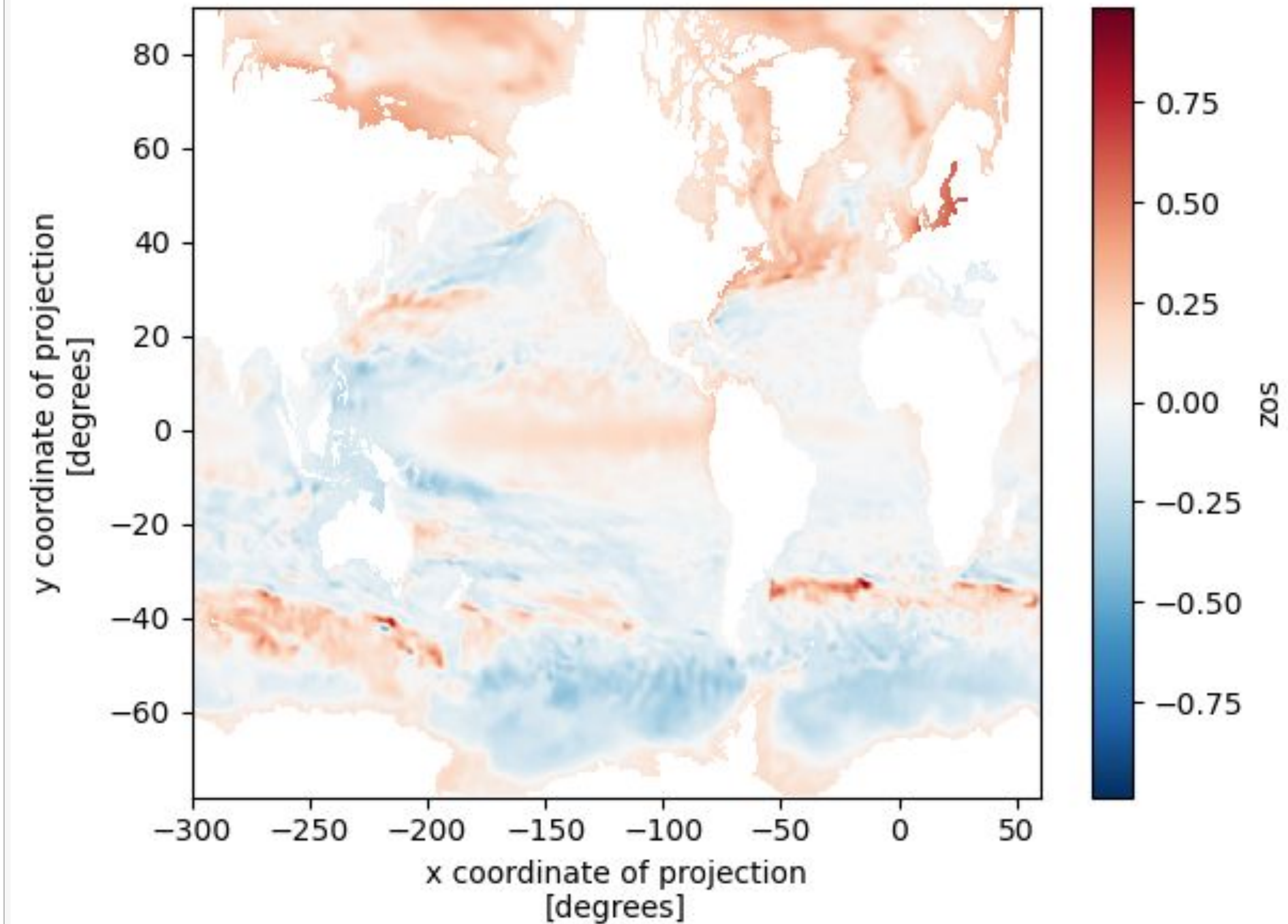


Zarr Workloads

- Read metadata from storage
- Read data from storage
- Decode bytes as an array
- Perform the computation

Zarr Example

```
>>> ds = xr.open_zarr(
...     "gs://cmip6/CMIP6/ScenarioMIP/NOAA-GFDL/...",
...     consolidated=True,
... )
>>> zos_2015jan = ds.zos.sel(time="2015-01-16")
>>> zos_2100dec = ds.zos.sel(time="2100-12-16")
>>> sealevelchange = zos_2100dec - zos_2015jan
>>> sealevelchange.plot.imshow()
```



GPU-Accelerated Zarr

Zarr natively supports NVIDIA GPUs

```
>>> import zarr
>>> zarr.config.enable_gpu()
>>> z = zarr.open_array("path/to/store.zarr")
>>> type(z[:])
cupy.ndarray
```

GPU Background

- Massively parallel processors. Extremely fast for the right problems.
- Often have separate memory (“device” memory)
 - Relatively fast compared to host memory (e.g. ~3 TB/s for an NVIDIA H100 GPU)
 - Relatively small compared to host memory (e.g. ~80 GB for an NVIDIA H100 GPU)
 - Relatively slow interconnect between host and device (~128 GB/s over PCIe Gen5)
- Programming GPUs well requires keeping the GPU busy
- CUDA **kernels** are essentially functions that run on the GPU
 - We will *not* talk about writing kernels here

GPU Programming

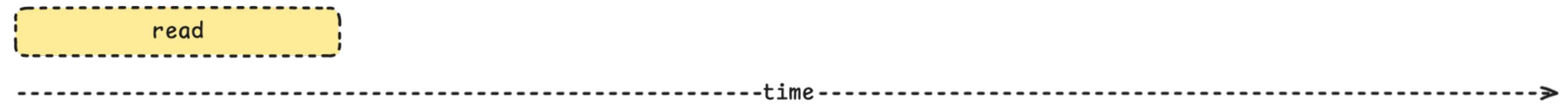
How does this run on the CPU?

```
>>> # On the CPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
```

GPU Programming

How does this run on the CPU? (1)

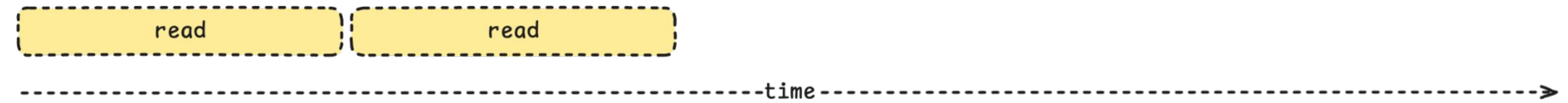
```
>>> # On the CPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
```



GPU Programming

How does this run on the CPU? (2)

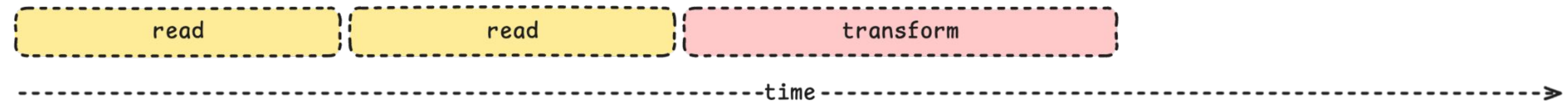
```
>>> # On the CPU  
>>> a0 = read_chunk("path/to/a")  
>>> b0 = read_chunk("path/to/b")  
>>> a1 = transform(a0)  
>>> b1 = transform(b0)
```



GPU Programming

How does this run on the CPU? (3)

```
>>> # On the CPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
```



GPU Programming

How does this run on the CPU? (4)

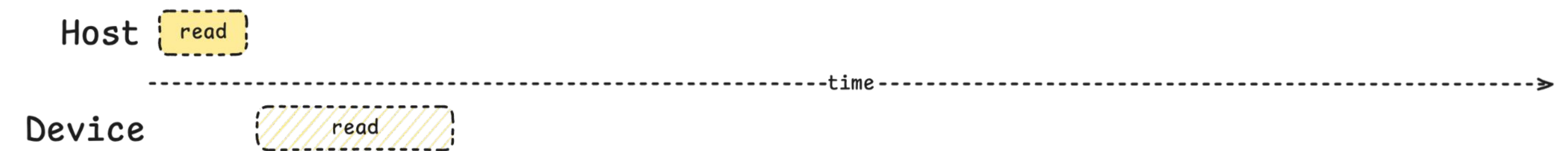
```
>>> # On the CPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
```



GPU Programming

How does this run on the GPU? (1)

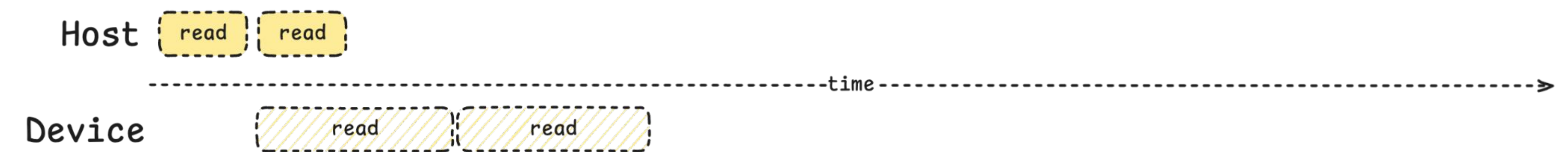
```
>>> # On the GPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
```



GPU Programming

How does this run on the GPU? (2)

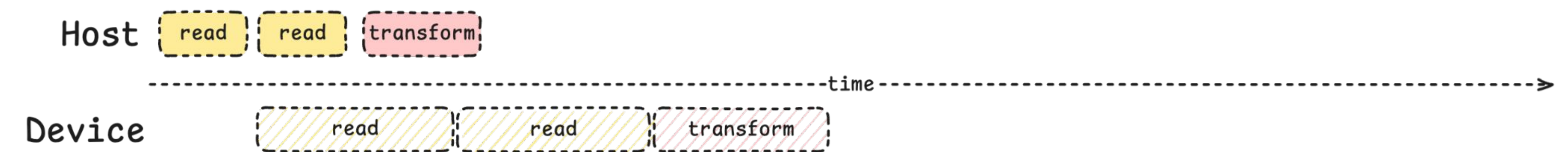
```
>>> # On the GPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
```



GPU Programming

How does this run on the GPU? (3)

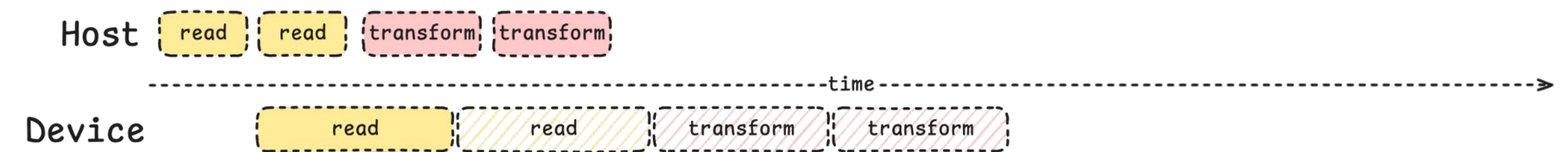
```
>>> # On the GPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
```



GPU Programming

How does this run on the GPU? (4)

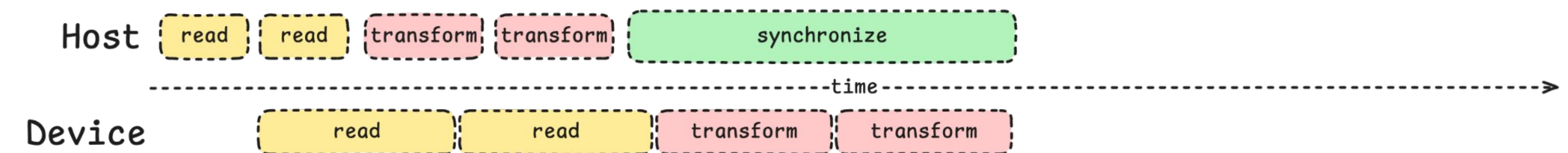
```
>>> # On the GPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
```



GPU Programming

How does this run on the GPU? (5)

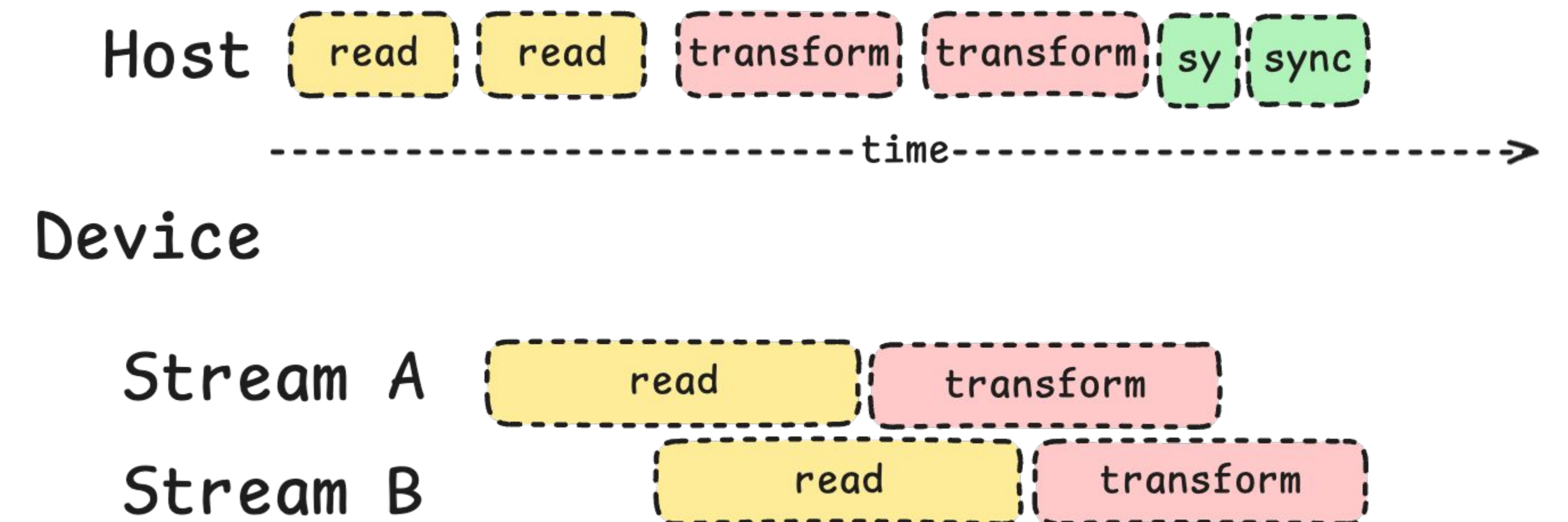
```
>>> # On the GPU
>>> a0 = read_chunk("path/to/a")
>>> b0 = read_chunk("path/to/b")
>>> a1 = transform(a0)
>>> b1 = transform(b0)
>>> stream.synchronize()
```



GPU Programming with Streams

For even more concurrency

```
>>> # On the GPU
>>> a0 = read_chunk("path/to/a", stream=a)
>>> b0 = read_chunk("path/to/b", stream=b)
>>> a1 = transform(a0, stream=a)
>>> b1 = transform(b0, stream=b)
>>> a.synchronize()
>>> b.synchronize()
```



GPU Programming

Summary

- Goal: queue up a bunch of work, keep the GPU busy
- Hide latency of launching GPU kernels
- At a lower level, non-blocking APIs and CUDA streams help with this

Sample Workload

Level	Shape	Size (MB)	Count / parent
Chunk	(256_000,)	1.024	400 chunks / shard
Shard	(102_400_000,)	409.6	8 shards / array
Array	(819_200_000,)	3,276.8	

- Zstd compression ratio: 5.3 (~77.5 MB /shard on disk)
- Load data, perform a bunch of matrix multiplications

Performance Summary

- Load each shard, do a bunch of matrix multiplications
- Zarr / NumPy uses zarr-python for I/O and decoding, NumPy for the matrix multiplication
- Zarr / CuPy uses zarr-python for I/O and decoding, CuPy for the matrix multiplication: **`zarr.config.enable_gpu()`**
- Custom / CuPy uses a hand-optimized I/O and decoding pipeline, nvcomp for GPU compression, CuPy for matrix multiplication.
- Caveats!
 - Deliberately tuned for the GPU
 - I haven't optimized the Zarr / NumPy or Zarr / CuPy implementations *at all*
- Conclusions?
 - GPUs *can* be very fast
 - One line-change gives a nice speedup
 - Still performance on the table for CPU and GPU

Implementation	Duration (ms)
Zarr / NumPy	19,892
Zarr / CuPy	3,407
Custom / CuPy	478

What makes the GPU fast?

- Custom I/O Pipeline:
 - Host / CPU to read a shard of bytes into host memory
 - Transfer bytes to device memory
 - Decode bytes into an output ndarray (nvcomp)
 - Perform a bunch of matrix multiplications (CuPy)
- Massive parallelism for decoding and compute
- Tight control over memory allocations and memory reuse
 - No large allocations on the critical path (host or device)
 - Could apply to CPU version too! See [zarr-python/2904](https://github.com/zarr-python/zarr/pull/2904)
- Use pinned host memory
 - Faster host to device / device to host transfers
- Overlapping I/O, Host to device transfer, Decode, and Compute tasks
 - Keep GPU utilization high
 - Use CUDA streams to express your available concurrency

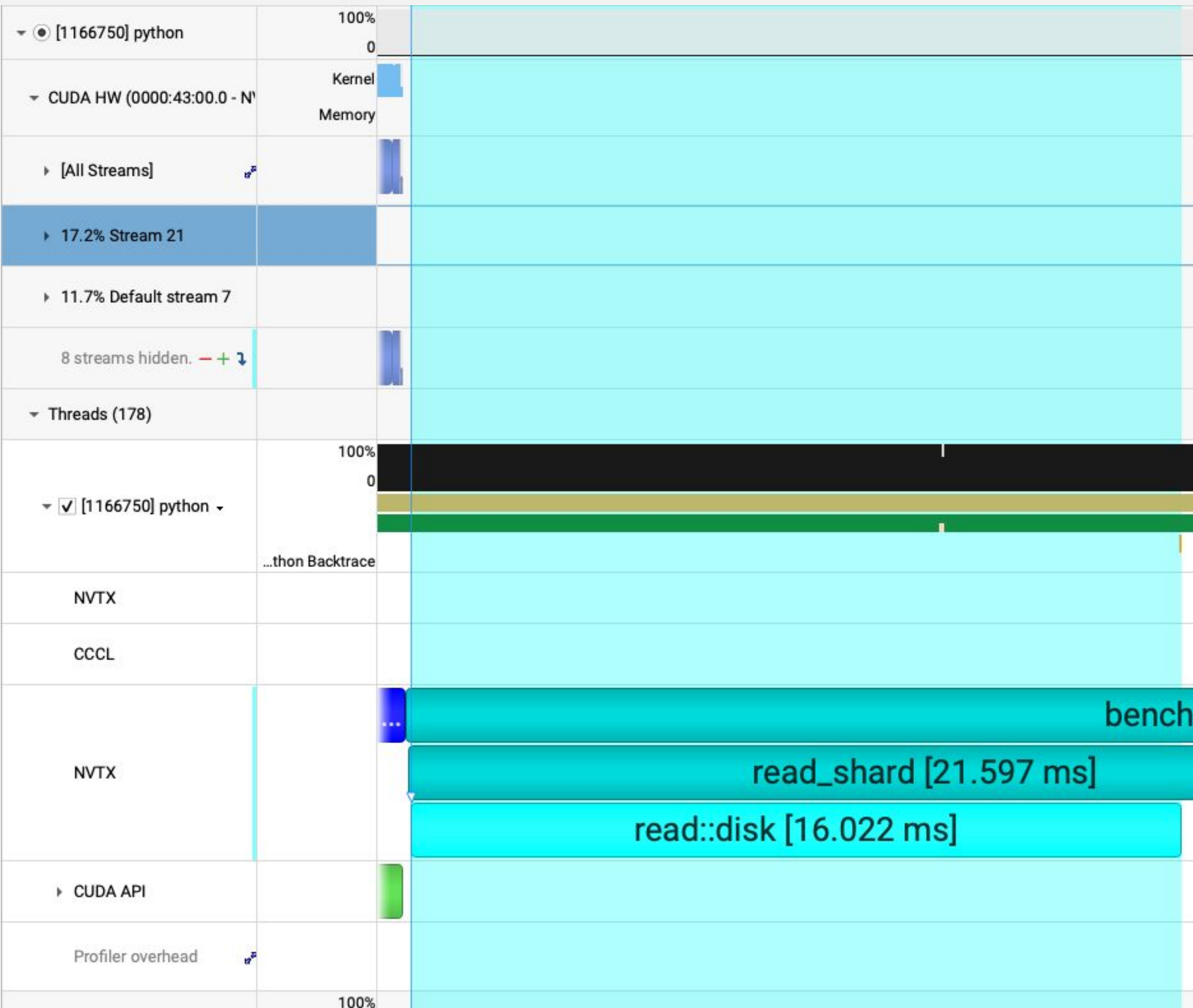
Reading Bytes

```
# outside the critical section
host_buffer = cupyx.empty_pinned(size, dtype="uint8")

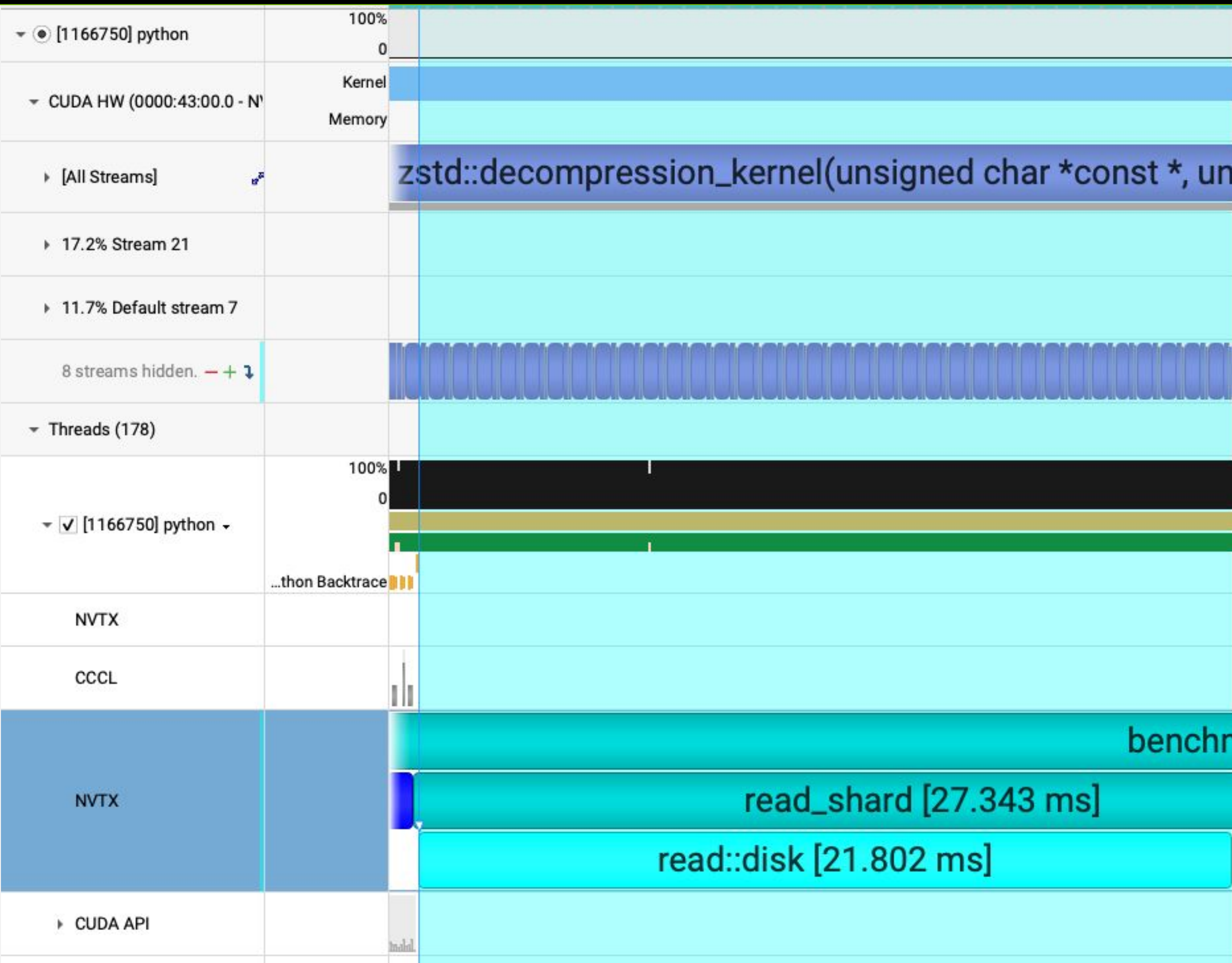
# inside the read_shard
with open(path, "rb") as f, nvtx.annotate("read::disk"):
    f.readinto(host_buffer)

# 1. No large (host) allocations
# 2. Blocking, so be fast or do in a thread pool
# 3. GPU is idle for the first shard, active for
#    subsequent
```

Shard 1



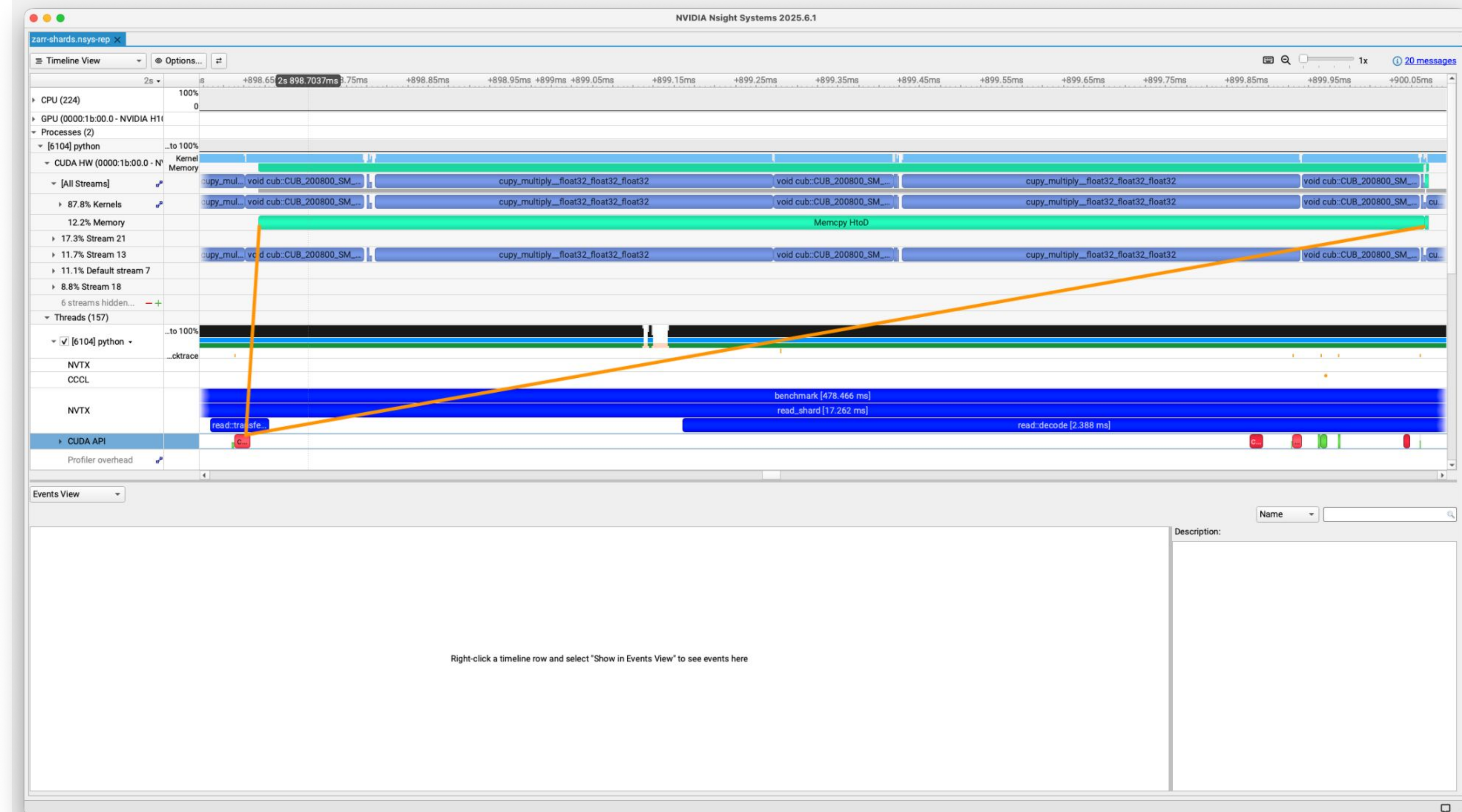
Shard 2



Transferring Bytes

```
with nvtx.annotate("read::transfer"), stream:  
    # device_buffer is a pre-allocated cupy.ndarray  
    device_buffer.set(  
        host_buffer[:idx].view(device_buffer.dtype),  
        stream=stream,  
    )
```

- # 1. No large (device) allocations
- # 2. non-blocking. Host is extremely fast
- # 3. Use pinned / page-locked memory for more perf
- # 4. Overlap with other compute

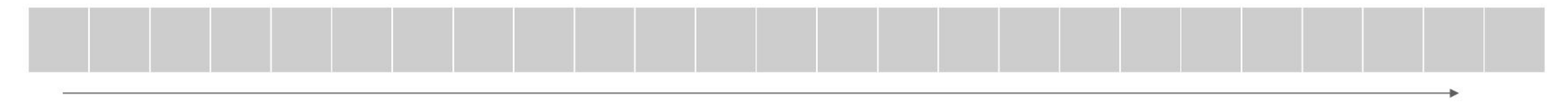


Decoding Bytes

```
zstd_codec = ZstdCodec(stream=stream)
device_arrays = [
    device_buffer[offset : offset + size]
    for offset, size in index
]
out_chunks = [o[slice(i * s, (i + 1) * s)] for i in idx]
with nvtx.annotate("read::decode"):
    zstd_codec.decode_batch(
        device_arrays,
        out=out_chunks
    )
```

- # 1. Custom, non-blocking nvcomp wrapper
- # 2. Decode directly into output buffer

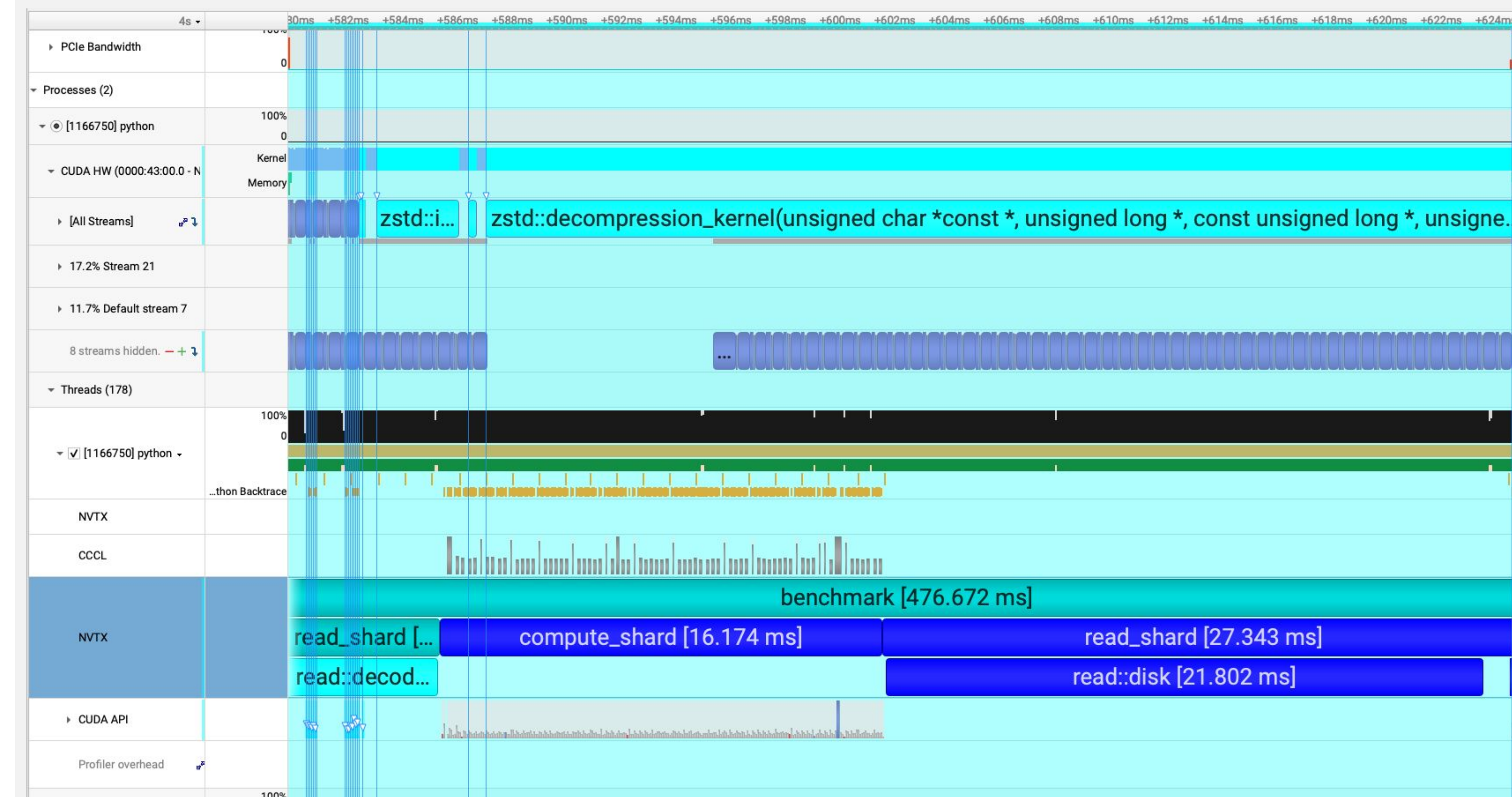
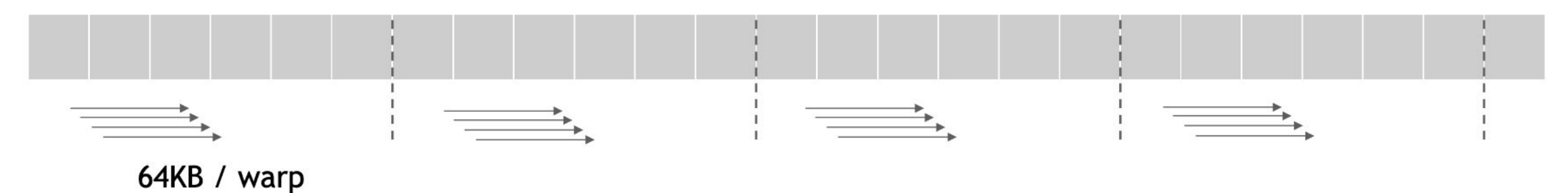
one LZ4 stream - difficult to map to the GPU, creates dependencies between copies, future copies depend on previous data



many smaller independent LZ4 streams - better mapping to the GPU, more concurrent tasks; need to store offsets/sizes



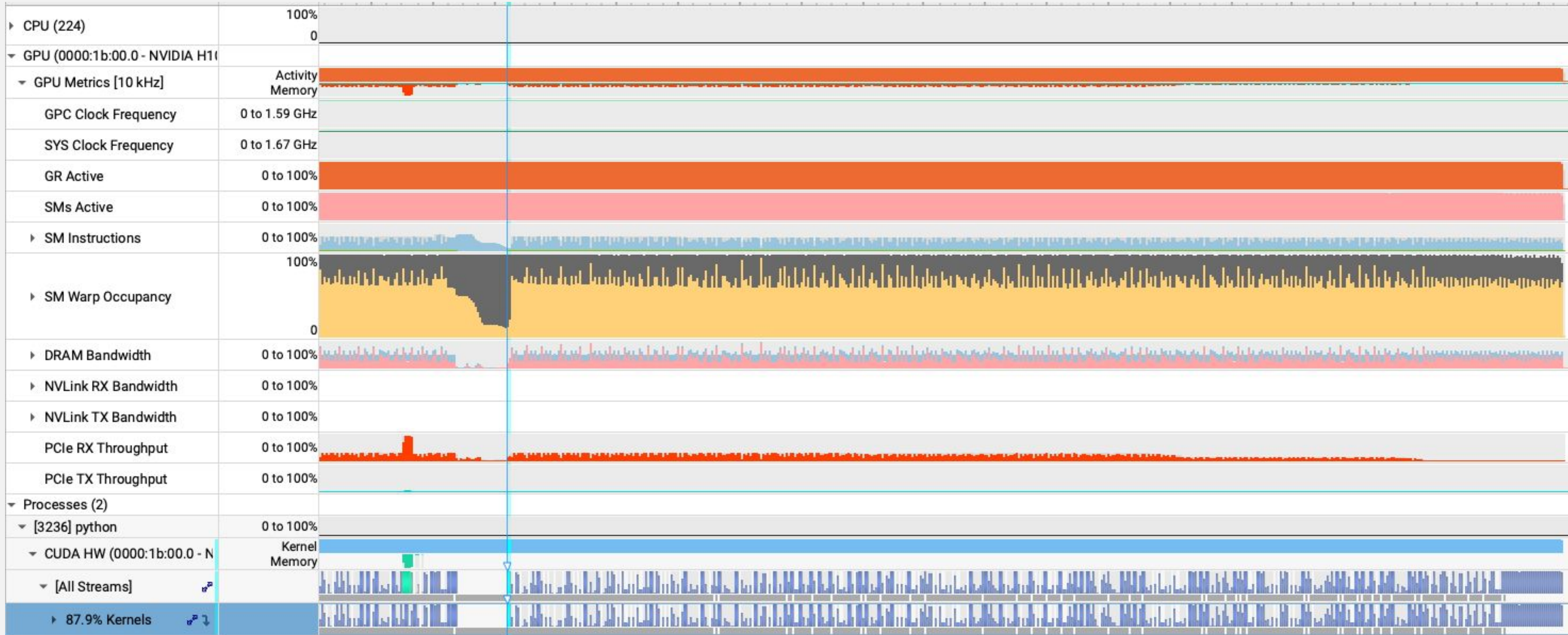
warp or threadblock-level parallelism within a LZ4 stream - threads cooperating on writing the output buffer



Matrix Multiplication

```
with stream, nvtx.annotate("compute_shard"):  
    for i in range(n):  
        shard @ shard
```

Just using CuPy's matmul



Summary

- GPUs can be very fast
 - Understand your memory allocations
 - Use pinned memory for transfers
 - Use nvcomp for decompression
 - Use CUDA streams for more concurrency
- Hoping to bring these improvements to Zarr
- Code: <https://github.com/TomAugspurger/cuda-streams-sample>



Thanks!

toaugspurger@nvidia.com